

AI Interviewer — Improvement Plan

Scope: All changes are confined to the interview subsystem (`src/app/interview/**`, `src/app/api/interview/**`, `src/server/infrastructure/security/interview-token.ts`, and `ai_interviewer_backend/**`). None of it touches candidates, jobs, matching, analytics, ambassador, or the rest of TalentHub, which is working correctly. Run `npm tsc --noEmit` after each phase.

Root-cause summary

The token / "broken system" problems (#3, #4, #7) share **three root causes**:

1. **TTL too short & minted too early.** `createInterviewRuntimeToken` mints with `ttlSeconds = 60 * 10 (10 min)` in `src/server/infrastructure/security/interview-token.ts`, and the token is created at *session creation* in `src/app/api/interview/session/route.ts` — **before** the candidate grants camera permissions and reads Q1. Meanwhile the frontend window is `TOTAL_SECONDS = 15 * 60 (15 min)` in `src/app/interview/[token]/page.tsx`. The token therefore dies mid-interview; setup overhead explains the observed ~4.5 min of usable time. `verifyInterviewRuntimeToken` then throws "Interview runtime token expired".
 2. **In-memory backend sessions.** `InterviewSessionManager.sessions` is a plain in-process dict (`ai_interviewer_backend/ai_interviewer.py`). On any FastAPI restart / cold-start (free-tier spin-down), `process_turn` raises `ValueError("Session not found")` → 404 → the interview looks broken.
 3. **Blocking evaluation.** `finishInterview` only sets `evaluation` if the backend returns it; if the eval call hangs or errors, the UI spins on "Processing evaluation..." forever (`src/app/interview/[token]/page.tsx`).
-

Plan per item

1 · Camera "fit-in" not zoom-in — *trivial, safe*

- `src/app/interview/[token]/page.tsx` (~line 1009): change `object-cover` → `object-contain` on the `<video>`, and give the container a fixed aspect (e.g. `aspect-video`) with `bg-black` so letterboxing looks intentional.
- **Files:** 1 line. **Risk:** none.

2 · More focused questions — *prompt-only, safe*

- Tighten `INTERVIEW_GUARDRAILS_PROMPT` / `INTERVIEW_FLOW_PROMPT` in `ai_interviewer_backend/ai_interviewer.py`: force *drill-down on one topic* (don't hop between projects), forbid multi-part questions, cap question length. The scope addenda already exist — strengthen "every question tests the single target skill."
- Optionally lower question `max_tokens` and reduce `depth_level` breadth.
- **Files:** Python prompts only. **Risk:** none to TalentHub (backend-isolated); validate by test interviews.

3 + 7 · Token expiry & splitting token from memory — *core fix*

Decouple the **short-lived auth token** from the **long-lived interview identity / memory**:

- **Raise TTL + sliding refresh.** Bump default TTL to ~30 min AND add a small `POST /api/interview/realtime/refresh` that — using the *still-valid Supabase auth session* — re-mints the runtime token and updates `signed_token_hash` + `token_expires_at`. Frontend rotates the token on each turn (or on a timer) transparently.
- **Graceful re-mint instead of scary error.** Since every route already re-checks the Supabase user, treat an expired runtime token as *re-issuable* rather than fatal — never surface "token expired" to the candidate.
- **Persist / re-hydrate memory (the "split").** Transcript turns are *already* saved to `interview_transcript_turns`. Make the backend stateless-friendly: when `process_turn` hits `Session not found`, **rehydrate** the Python session from the persisted transcript + candidate profile instead of 404-ing. This removes cold-start breakage.
- **Files:** `interview-token.ts`, `session/route.ts`, new refresh route, `realtime/turn` route, `page.tsx` (token rotation + retry-once), `ai_interviewer.py` (rehydrate path).
- **Risk:** medium — confined to interview flow; gate behind test interviews.

4 + 8 · "Stuck at processing" → async evaluation agent — *robustness*

- Make evaluation **non-blocking**: `terminate` / `should_end` returns immediately with `status: "EVALUATING"` and persists a pending `assessment_results` row; run the evaluator in the background (Next.js `after()` now, or a dedicated worker / "separate agent" later for extended-thinking deep eval). Frontend **polls** `/api/interview/results` and shows a friendly "Evaluating..." state.
- **Hard timeout + fallback:** if eval exceeds N seconds or errors, fall back to the existing `DEFAULT_EVALUATION` ("HR will review") so the candidate never hangs. The `DEFAULT_EVALUATION` scaffold already exists in `terminate/route.ts`.
- Item 8's "separate agent / extended thinking" fits here: the async worker can use a slower, higher-quality model without blocking the candidate.
- **Files:** `terminate` route, `results` route, `page.tsx` polling.
- **Risk:** medium; the fallback path guarantees no worse than today.

5 · Failing without justification — *guarantee + surface*

- The evaluator prompt already *requires* `pass_fail_justification` + evidence on FAIL (`ai_interviewer_backend/evaluator.py`). Two gaps: (a) malformed-JSON fallbacks produce generic rationale; (b) the UI shows the decision prominently but rationale only if present.
- **Fix:** harden evaluator JSON parsing with a retry; **never emit a FAIL without a non-empty rationale.final + at least one evidence entry** (validate server-side, downgrade to REVIEW if missing). Ensure the results UI always renders the justification for a FAIL.
- **Files:** `evaluator.py`, `terminate` route validation, `page.tsx`.
- **Risk:** low.

6 · Language assessment — explainability + easier — *UX + scoring*

- **Explainability:** the backend already returns six sub-scores (grammar / vocab / fluency / listening / writing) but the results view only prints the overall CEFR level (`page.tsx`). Render the **per-dimension**

breakdown + the rationale so a fail / level is explained.

- **Easier on the interviewee:** clarify task instructions (the listening + writing-dictation phases are the stress points), make the pass bar transparent up-front (B1+), and soften dictation penalties / allow a repeat of the listening passage. Add encouraging framing.
 - **Files:** `page.tsx` (breakdown UI), `ai_interviewer.py` (task instructions / repeat), `evaluator.py` (scoring leniency + threshold clarity).
 - **Risk:** low.
-

Suggested sequencing (safest → most involved)

- **Phase 1 (quick wins, near-zero risk):** #1 camera, #2 prompts, #5 justification guarantee, #6 explainability UI.
- **Phase 2 (reliability):** #3 TTL raise + silent re-mint, #4 async eval + timeout fallback.
- **Phase 3 (architecture):** #7 memory rehydration / stateless backend, #8 separate deep-eval agent.

Everything is backend-isolated or interview-scoped, so TalentHub's working surface stays intact. `npx tsc --noEmit` after each phase.

Open questions before implementation

1. Where is the FastAPI backend hosted (Render / Railway / other)? This decides whether cold-starts are the memory culprit and whether a background worker (#8) is feasible.
2. For #3, prefer the **simple** fix (raise TTL to ~30 min as a stopgap) or go straight to **sliding refresh + rehydration** (#7)?
3. For #6, OK to **lower language difficulty / penalties**, or keep the bar and only improve *explainability*?